《操作系统综合实验》 实验报告

实验名称	Linux 下多进程编程	实验序号	2
实验日期	2025.5.15	实验人	魏朝旭

一、 实验题目

问题一 UNIX Shell 和历史记录

该项目包括设计一个 C 程序作为一个 shell 接口，它接受用户命令，然后在单独的进程中执行每个命令。这个项目可以在任何 Linux、UNIX 或 Mac OS X 系统上完成。

shell 界面给用户一个提示，然后输入下一个命令。下面的示例说明了提示符 osh> 和用户的下一个命令：cat prog.c 。（此命令使用 UNIX cat 命令在终端上显示文件 prog.c。）

实现 shell 接口的一种技术是让父进程首先读取用户在命令行中输入的内容（在本例中为 cat prog.c），然后创建一个单独的子进程来执行该命令。除非另有说明，否则父进程在继续之前等待子进程退出。这在功能上类似于图 3.10 中所示的新流程创建。但是，UNIX shell 通常还允许子进程在后台或同时运行。为此，我们在命令末尾添加一个与号 (&)。因此，如果我们将上面的命令重写为 osh> cat pro.c &父进程和子进程将同时运行。

单独的子进程是使用 fork() 系统调用创建的，用户的命令是使用系统调用 exec() 系列之一执行的（如第 3.3.1 节所述）。

用户在 osh> 提示符下输入命令 ps -ael，存储在 args 数组中的值是：

Args[0] = “ps”

Args[1] = “-ael”

Args[2] = NULL

这个 args 数组将被传递给 execvp() 函数，它具有以下原型：

execvp (char *command, char *params[]) ;

这里，command 表示要执行的命令，params 存储此命令的参数。对于这个项目，execvp() 函数应该被调用为 execvp(args [0], args)。请务必检查用户是否包含 & 以确定父进程是否要等待子进程退出。

问题二 创建历史任务

下一个任务是修改 shell 接口程序，使其提供 history 功能，允许用户访问最近输入的命令。通过使用该功能，用户最多可以访问 10 个命令。命令会从 1 开始连续编号，超过 10 会继续编号。例如，如果用户输入了 35 条命令，则最近的 10 条命令将编号为 26 到 35。

History 6 ps

5 ls -l

4 top

3 cal

2 who

1 date

1、当用户输入 !! 执行历史记录中的最新命令。

2、当用户输入单 ！后跟整数 N，执行历史记录中的第 N 个命令。

该程序还应该管理基本的错误处理。

1、如果历史中没有命令，输入 ！！应该会产生一条消息“No commands in history”。

2、如果没有与用单个 ！输入的数字对应的命令，程序应该输出“No such command in history”。

二、 相关原理与知识

问题一：

1、进程管理： 在操作系统中，进程是程序执行的实例。每个进程都有自己的地址空间、程序代码和数据。在这个问题中，程序使用 fork() 系统调用创建一个新的进程来执行用户输入的命令，从而实现了进程的管理。

2、系统调用： 程序使用了多个系统调用来与操作系统进行交互，比如

fork()、execvp()、waitpid() 等。这些系统调用使得程序能够访问操作系统提供的功能，比如创建新进程、执行程序、等待进程退出等。

3、fork()函数：是一个在 Unix-like 系统中常见的系统调用，用于创建一个新的进程，称为子进程，该子进程是调用进程（称为父进程）的一个副本。fork() 函数在调用成功后，在父进程中返回子进程的 PID（进程标识符），而在子进程中返回 0。

问题二：

问题二与问题一相比主要涉及到代码的变化，涉及到的操作系统相关知识原理与问题一一致。

三、 实验过程

问题一：

1、读取用户输入：首先 ptintf 打印提示符，然后使用 fgets 函数从标准输入读取用户输入的命令行，并存储在 buffer 数组中。

2、分割命令行：使用 split_command 函数将用户输入的命令行按空格分割成命令和参数，并存储在 args 数组中。

3、执行命令：使用 fork 函数创建一个子进程，并在子进程中使用 execvp 函数执行用户输入的命令。如果用户输入的命令包含 &，则表示是后台命令，子进程会在后台执行。

4、等待子进程结束： 在父进程中，如果用户输入的命令不是后台命令，则使用 waitpid 函数等待子进程结束，并获取子进程的退出状态。如果是后台命令，则父进程继续等待下一个命令输入。

5、循环处理下一个命令： 回到第一步，继续等待用户输入下一个命令

问题二：

1、在问题一代码基础上加以修改增添功能得到问题二

2、定义历史记录结构体：首先，我们定义了一个结构体 History 来存储每个历史命令的内容和编号。

3、新增历史命令编号变量：我们新增了一个变量 history_number 来跟踪当前

历史命令的编号。

4、保存命令到历史记录：在 add_to_history()函数中，我们将每个新的命令保存到历史记录数组中，并为其分配一个递增的编号。

5、执行历史命令：在 execute_history_command()函数中，我们根据输入的命令执行历史记录中的命令。如果是!!命令，则执行最新的命令；如果是!N 命令，则执行指定编号的命令。

6、主函数修改：在主函数中，我们相应地调整了输入的命令的处理逻辑，以便正确地调用历史命令的执行函数。并增添了错误处理机制。

四、 实验结果与分析

问题一：

实验运行结果

```
liao@liao-PC:~/Desktop$ gcc -o main main.c
liao@liao-PC:~/Desktop$ ./main
osh> ps -ael
F S   UID     PID    PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
4 S     0       1       0  0  80   0 -  5647 -      ?        00:00:05 systemd
1 S     0       2       0  0  80   0 -     0 -      ?        00:00:00 kthreadd
1 I     0       3       2  0  60 -20 -     0 -      ?        00:00:00 rcu_gp
1 I     0       4       2  0  60 -20 -     0 -      ?        00:00:00 rcu_par_gp
1 I     0       5       2  0  60 -20 -     0 -      ?        00:00:00 slub_flushwq
1 I     0       6       2  0  60 -20 -     0 -      ?        00:00:00 netns
1 I     0       8       2  0  60 -20 -     0 -      ?        00:00:00 kworker/0:0H-events_highpri
1 I     0      10       2  0  60 -20 -     0 -      ?        00:00:00 mm_percpu_wq
1 I     0      11       2  0  80   0 -     0 -      ?        00:00:00 rcu_tasks_kthread
1 I     0      12       2  0  80   0 -     0 -      ?        00:00:00 rcu_tasks_rude_kthread
1 I     0      13       2  0  80   0 -     0 -      ?        00:00:00 rcu_tasks_trace_kthread
1 S     0      14       2  0  80   0 -     0 -      ?        00:00:00 ksoftirqd/0
1 I     0      15       2  0  80   0 -     0 -      ?        00:00:02 rcu_preempt
1 S     0      16       2  0 -40   - -     0 -      ?        00:00:00 migration/0
1 S     0      17       2  0   9   - -     0 -      ?        00:00:00 idle_inject/0
1 S     0      19       2  0  80   0 -     0 -      ?        00:00:00 cpuhp/0
1 S     0      20       2  0  80   0 -     0 -      ?        00:00:00 cpuhp/1
1 S     0      21       2  0   9   - -     0 -      ?        00:00:00 idle_inject/1
1 S     0      22       2  0 -40   - -     0 -      ?        00:00:00 migration/1
1 S     0      23       2  0  80   0 -     0 -      ?        00:00:00 ksoftirqd/1
```

```
osh> cat main.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/stat.h>
#include <fcntl.h>

#define Max_Line 1024

void split_command(char *input, char **args) {
    char *token;
    char *delim = " ";
    int i = 0;

    while ((token = strtok(input, delim)) != NULL) {
        if (i > 0 && token[0] == '"' && token[strlen(token) - 1] == '"') {
            // Remove the quotes from the token
            token[strlen(token) - 1] = '\0'; // r��ĭβ������
            args[i++] = token + 1; // ������и������
        } else {
```

```
osh> echo 'hello world'
'hello world'
Command exited with status 0
```

结果解释

1、首先对 main 程序进行编译然后运行

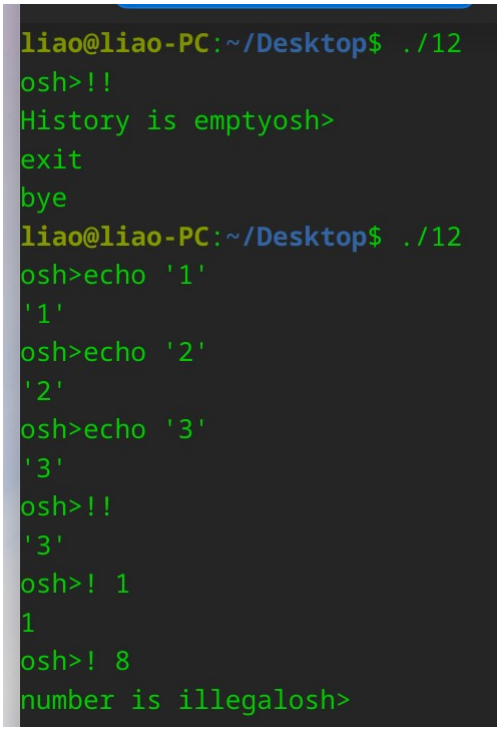
2、第一张截图输入命令 ps -ael 显示当前系统运行进程的详细信息。

3、第二张截图输入命令 cat main.c 在终端上显示文件内容

4、第三章截图输入命令 echo ‘hello world’ 在终端打印字符串’hello world’

问题二：

实验运行结果



结果解释

1、最开始运行程序，输入 !! 显示没有历史命令，然后 exit 退出

2、连续输入 3 个 echo 命令，然后输入 !! 执行最新的一条命令打印 '3' 。

3、输入 ! 1 执行 echo '1' 命令，输入 ! 8 则会超出输入命令的数量，抛出异常。

五、 问题总结

1、**问题：**解析用户输入的命令行，并将命令和参数分离，这可能涉及到字符串操作和处理引号等特殊字符的情况。

解决办法：使用字符串操作函数（如 strtok()和 strcmp()）来分割和比较命令行输入，同时考虑特殊情况（如引号内的空格）。可以编写一个函数来实现这一步骤，确保正确地解析命令和参数。

2、**问题：**在问题二中，需要设计和实现一个历史记录管理系统，用于记录和管理用户输入的命令历史。涉及到如何存储历史记录、按照规则管理历史记录以及如何检索和使用历史记录。

解决办法：借用结构体将命令行顺序和内容绑定，同时设计合适的数据结构数组来存储历史记录，并实现相应的函数来添加、显示和执行历史

命令。

3、问题：一些小细节，如异常机制、用户交互、提示信息等容易忽略。

解决方法：多添加测试数据去完善程序可能会存在的漏洞。

六、 源代码

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define Max_Line 1024
#define Max_History 10

// 历史记录结构体
typedef struct {
    char command[Max_Line];
    int number;
} History;

// 历史记录数组
History history[Max_History];
int history_index = 0; // 当前历史命令的数量
int history_number = 0; // 当前历史命令的编号

// 分割命令函数
void split_command(char *input, char **args) {
    char *token;
    char *delim = " ";
```

```
    int i = 0;

    while ((token = strtok(input, delim)) != NULL) {
        if (i > 0 && token[0] == '"' && token[strlen(token) - 1] == '"') {
            // Remove the quotes from the token
            token[strlen(token) - 1] = '\0'; // 删除末尾的引号
            args[i++] = token + 1; // 跳过开头的引号
        } else {
            args[i++] = token;
        }
        input = NULL; // strtok 设置为 NULL，以便从上一次调用的位置开始搜索下一个 token
    }
    args[i] = NULL;
}

// 执行命令
void execute_command(char **args) {
    pid_t pid = fork();
    if (pid == -1) {
        perror("fork");
        exit(EXIT_FAILURE);
    } else if (pid == 0) {
        // 子进程
        if (execvp(args[0], args) == -1) {
            perror("execvp");
            exit(EXIT_FAILURE);
        }
    } else {
```

```
            // 父进程
            int status;
            waitpid(pid, &status, 0);
            if (WIFEXITED(status)) {
                printf("Command exited with status %d\n", WEXITSTATUS(status));
            }
        }
}

// 保存命令到历史记录
void add_to_history(char *command) {
        if (history_index < Max_History) {
            strcpy(history[history_index].command, command);
            history[history_index].number = ++history_number;
            history_index++;
        } else {
            // 如果历史记录已满，覆盖最早的命令
            history[0] = history[history_index - 1];
            history[0].number = ++history_number;
            history_index = Max_History;
        }
}

// 显示历史记录
void show_history() {
        if (history_index == 0) {
            printf("No commands in history\n");
            return;
        }
        printf("History:\n");
```

```
    for (int i = history_index - 1; i >= 0; i--) {
        printf("%d: %s\n", history[i].number, history[i].command);
    }
}

// 执行历史命令
void execute_history_command(char *input) {
    char *num = strtok(input + 1, " ");
    if (num == NULL) {
        // 执行最新命令
        if (history_index > 0) {
            char *command_args[Max_Line/2+1];
            split_command(history[history_index - 1].command, command_args);
            execute_command(command_args);
        } else {
            printf("No commands in history\n");
        }
    } else {
        int index = atoi(num);
        if (index > 0 && index <= history_index) {
            // 执行指定编号的命令
            char *command_args[Max_Line/2+1];
            split_command(history[index - 1].command, command_args);
            execute_command(command_args);
        } else {
            // 无效的命令编号
            printf("No such command in history\n");
        }
    }
}
```

```
int main() {
    char buffer[Max_Line];
    char *args[Max_Line/2+1];

    while (1) {
        printf("osh> ");
        fgets(buffer, Max_Line, stdin);
        buffer[strcspn(buffer, "\n")] = 0;

        if (strcmp(buffer, "exit") == 0) {
            break;
        } else if (strncmp(buffer, "!!", 2) == 0) {
            // 执行最新命令
            execute_history_command(buffer);
        } else if (strncmp(buffer, "!", 1) == 0) {
            // 执行指定编号的命令
            execute_history_command(buffer);
        } else {
            // 分割命令并执行
            split_command(buffer, args);
            execute_command(args);

            // 保存命令到历史记录
            add_to_history(buffer);
        }
    }

    return 0;
}
```